



รายงานโครงงานวิชาการเทคโนโลยีกลุ่มเมฆ (Cloud Technology)

ภาคเรียนที่ 1 ปีการศึกษา 2569

โครงงานระบบจำลองห้องเล่นและวิเคราะห์เหตุการณ์แบบเรียลไทม์บนคลาวด์

ระบบประมวลผลแยกส่วน SNS Fan-out, SQS DLQ, ElastiCache Cache-Aside

และตัวขับเคลื่อนกฎกติกาเกมแบบ Pure Function

(Disney Lorcana PlayLab Cloud: Decoupled Event Processing,

In-Memory Caching & Pure Game Rule Engine)

กลุ่ม G21

รายงานฉบับสมบูรณ์ (Stage 3: ผลการทดลอง ข้อจำกัด ค่าใช้จ่าย และการขยายระบบ)

คณะผู้จัดทำ:

1. นายชยุต บุญวัฒน์	รหัสนักศึกษา 67070032
2. นายธนัทภัทร พรหมทอง	รหัสนักศึกษา 67070069
3. นายภูริ ประชาสุขสิน	รหัสนักศึกษา 67070137
4. นางสาววรรณนิศา อมรวงศ์ไพบุลย์	รหัสนักศึกษา 67070155
5. นายอสิรา พุ่มดอกไม้	รหัสนักศึกษา 67070199
6. นายวรวิชัย คงทอง	รหัสนักศึกษา 67070275

อาจารย์ประจำวิชา:

1. ดร. ธนานพ ทองถาวร
2. ผศ.ดร. พัฒนพงษ์ ฉันทมิตรโอภาส
3. ผศ.ดร. ลภัส ประดิษฐ์ทัศน์

คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ตุลาคม 2569

บทคัดย่อภาษาไทย

รายงานฉบับนี้นำเสนอผลการทดลอง ข้อจำกัด และค่าใช้จ่ายของระบบ **Disney Lorcana PlayLab Cloud (กลุ่ม G21)** ในระยะที่ 3 (Stage 3) ซึ่งต่อยอดจากสถาปัตยกรรม Lean Multi-AZ VPC, Application Load Balancer และ EC2 Auto Scaling Group ที่พัฒนาเสร็จในระยะที่ 2 โดยเพิ่มส่วนประกอบใหม่ 5 ส่วน ได้แก่ (1) ระบบ Content Caching ด้วย hash-based immutable assets และ gzip compression ลดขนาดการรับส่งไฟล์ข้อมูลการลดลง 81.43% (2) ระบบ Dead Letter Queue (DLQ) สำหรับจัดการข้อความที่ประมวลผลล้มเหลวใน SQS (3) ระบบกระจายผลการแข่งขันผ่าน Amazon SNS Fan-out ไปยัง SQS 2 คิวคู่ขนานสำหรับบันทึกประวัติและคำนวณสถิติแยกอิสระ (4) ระบบตารางผู้นำพร้อม ElastiCache Valkey แบบ Cache-Aside ที่ลดเวลาประมวลผลฝั่ง server จาก 41 มิลลิวินาที เหลือ 1 มิลลิวินาที (ค่ามัธยฐาน) และ (5) ตัวขับเคลื่อนกฎกติกาเกมแบบ Pure Function ที่รองรับคีย์เวิร์ดทางการ 14 ชนิด ระบบ Ability 10 ประเภท การ Shift, Location, Song และ Item พร้อมระบบเลือกเป้าหมายผ่าน Prompt

การทดลองทั้ง 10 หัวข้อบน AWS Academy Learner Lab (us-east-1) ครอบคลุมการเล่นข้ามเครื่อง ความปลอดภัย Failover, Auto Scaling, Infrastructure as Code, Content Caching, DLQ Retry, SNS Fan-out, ElastiCache Cache-Aside และ Ability Coverage จากข้อมูลการวัด 3,242 ใบ ค่าใช้จ่ายจริงจาก AWS Billing ในเดือนกันยายน 2569 อยู่ที่ 10.29 ดอลลาร์สหรัฐ (1-27 กันยายน) โดย Application Load Balancer เป็นค่าใช้จ่ายหลัก (69%)

คำสำคัญ: คลาวด์คอมพิวติง, SNS Fan-out, SQS Dead Letter Queue, ElastiCache Cache-Aside, Game Rule Engine, Multi-AZ VPC, Auto Scaling, AWS Learner Lab

Abstract

This report presents the experimental results, limitations, and cost analysis of the **Disney Lorcana PlayLab Cloud (Group G21)** system in Stage 3. Building upon the Lean Multi-AZ VPC, Application Load Balancer, and EC2 Auto Scaling Group architecture delivered in Stage 2, this stage introduces five new subsystems: (1) Content Caching with hash-based immutable assets and gzip compression, reducing card data transfer by 81.43%; (2) Dead Letter Queue (DLQ) for handling failed SQS messages with a 3-retry redrive policy; (3) Amazon SNS Fan-out for distributing match results to two parallel SQS queues serving independent match history and player statistics consumers; (4) Leaderboard with ElastiCache Valkey using Cache-Aside pattern, reducing the median server-side processing time from 41 ms to 1 ms; and (5) a Pure Function Game Rule Engine supporting 14 official keywords, 10 ability effect types, Shift, Location, Song, and Item mechanics with prompt-based target selection.

Ten experiments were conducted on AWS Academy Learner Lab (us-east-1), covering cross-instance gameplay, security hardening, failover recovery (2.8–3.9 minutes), CPU-based auto scaling with Node.js cluster mode, Infrastructure as Code, content caching, DLQ retry resilience, SNS fan-out decoupling, ElastiCache cache-aside performance, and ability coverage analysis across 3,242 cards (Set 1 through Set 13). Actual AWS billing for September 2569 (1–27 September) totaled \$10.29, with Application Load Balancer accounting for 69% of the cost.

Keywords: Cloud Computing, SNS Fan-out, SQS Dead Letter Queue, ElastiCache Cache-Aside, Game Rule Engine, Multi-AZ VPC, Auto Scaling, AWS Learner Lab

กิตติกรรมประกาศ

โครงการ Disney Lorcana PlayLab Cloud สำเร็จลุล่วงตามวัตถุประสงค์ของการส่งมอบงานในระยะที่ 3 (Stage 3) ได้ด้วยความอนุเคราะห์ คำแนะนำ และการชี้แนะทางวิชาการจากคณาจารย์ประจำรายวิชาการเทคโนโลยีกลุ่มเมฆ (Cloud Technology) คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง ได้แก่:

1. ดร. ธนเทพ ทองถาวร
2. ผศ.ดร. พัฒนพงษ์ ฉันทมิตรโอภาส
3. ผศ.ดร. ลภัส ประดิษฐ์ทัศน์

ที่ได้กรุณาถ่ายทอดองค์ความรู้ด้านสถาปัตยกรรมคลาวด์ การออกแบบระบบเครือข่าย Multi-AZ VPC ความปลอดภัย และการบริหารจัดการทรัพยากรอย่างคุ้มค่า ซึ่งเป็นรากฐานสำคัญในการออกแบบและแก้ปัญหาทางวิศวกรรมซอฟต์แวร์ในโครงการนี้

คณะผู้จัดทำขอขอบพระคุณเพื่อน ๆ นักศึกษาคณะเทคโนโลยีสารสนเทศทุกท่านที่ได้ร่วมทดสอบระบบ ให้ข้อคิดเห็นด้านประสิทธิภาพการใช้งาน และร่วมตรวจสอบข้อผิดพลาดของระบบ รวมถึงขอขอบคุณครอบครัวที่ให้การสนับสนุนและเป็นกำลังใจในการศึกษาค้นคว้าตลอดมา

คณะผู้จัดทำ โครงการกลุ่ม G21

ตุลาคม 2569

สารบัญ

บทคัดย่อภาษาไทย	i
Abstract	ii
กิตติกรรมประกาศ	iii
1 บทนำ	1
1.1 ความเป็นมาและความสำคัญ	1
1.2 วัตถุประสงค์	1
1.3 ขอบเขต	1
2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง	2
2.1 Amazon Simple Notification Service (SNS)	2
2.2 Amazon Simple Queue Service (SQS) และ Dead Letter Queue	2
2.3 Amazon ElastiCache และรูปแบบ Cache-Aside	2
2.4 Content Caching ด้วย Hash-Based Immutable Assets	2
2.5 Node.js Cluster Module	2
2.6 Pure Function Game Engine	2
3 การออกแบบระบบ	4
3.1 สถาปัตยกรรมระบบรวม	4
3.2 เหตุผลในการออกแบบ (Design Rationale)	4
3.2.1 Multi-AZ โดยไม่ใช้ NAT Gateway	4
3.2.2 Health Check ตรวจสอบเฉพาะ Node.js ไม่ตรวจสอบ DynamoDB	4
3.2.3 Node.js Cluster Mode สำหรับ t3.micro	5
3.2.4 SNS Fan-out แทนการเขียน DynamoDB โดยตรง	5
3.2.5 ElastiCache Node แทน Serverless	5
3.3 ตัวขับเคลื่อนกฎกติกาเกม	5
4 ผลการทดลอง	6

4.1	การทดลองที่ 1: การเล่นเกมออนไลน์ข้ามเครื่อง	6
4.2	การทดลองที่ 2: ความปลอดภัยบนระบบจริง	6
4.3	การทดลองที่ 3: Failover เมื่อ Backend ล่ม	6
4.4	การทดลองที่ 4: Auto Scaling	7
4.5	การทดลองที่ 5: Infrastructure as Code	7
4.6	การทดลองที่ 6: Content Caching (T02)	7
4.7	การทดลองที่ 7: Dead Letter Queue (T03)	8
4.8	การทดลองที่ 8: SNS Fan-out (T04)	8
4.9	การทดลองที่ 9: ElastiCache Cache-Aside (T05)	8
4.10	การทดลองที่ 10: Ability Coverage (T11)	9
5	ข้อจำกัดและค่าใช้จ่าย	10
5.1	ข้อจำกัดของระบบ	10
5.2	ประมาณการค่าใช้จ่าย	10
6	สรุปและข้อเสนอแนะ	12
6.1	สรุปผลการดำเนินงาน	12
6.2	ข้อเสนอแนะ	12
	บรรณานุกรม (References)	13

สารบัญตาราง

3.1	โครงสร้าง Module ของตัวขับเคลื่อนกฎกติกาเกม	5
4.1	ผลทดสอบความปลอดภัย (OWASP A02, A05, A07)	6
4.2	ผลการทดลอง Failover (หลักฐาน: failover_app_*.txt)	7
4.3	ผลการทดลอง Auto Scaling (หลักฐาน: stress_scaling_*.txt)	7
4.4	ผล Content Caching (หลักฐาน: caching_2026-09-29*.json)	7
4.5	ผล ElastiCache Leaderboard เป็นค่ามัธยฐานและ p95 (หลักฐาน: leaderboard_cache_2026-09-29T17-18-08-612Z.json)	8
4.6	ระดับการรองรับ Ability ของตัวขับเคลื่อนกฎกติกา	9
5.1	ประมาณการค่าใช้จ่ายรายชั่วโมง	10
5.2	ค่าใช้จ่ายจริงจาก AWS Billing (1-27 กันยายน 2569)	11

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญ

Disney Lorcana PlayLab Cloud เป็นเว็บแอปพลิเคชันสำหรับเล่นการ์ดเกม Disney Lorcana แบบออนไลน์ผ่านเบราว์เซอร์ โดยในระยะที่ 2 ได้พัฒนาสถาปัตยกรรมพื้นฐานประกอบด้วย Lean Multi-AZ VPC, Application Load Balancer, EC2 Auto Scaling Group และระบบห้องเล่นแบบเรียลไทม์ผ่าน API Gateway WebSocket ให้สามารถรองรับผู้เล่น 2 คนที่ ALB ส่งไปคนละเครื่องเล่นด้วยกันได้ พร้อมความทนทานต่อการล่มของเครื่องเดียว

ในระยะที่ 3 มุ่งเน้นการขยายระบบใน 3 ด้าน ได้แก่ (1) การแยกส่วนประมวลผลเหตุการณ์ (Event Decoupling) ด้วย Amazon SNS, SQS และ Dead Letter Queue (2) การเพิ่มประสิทธิภาพการตอบสนองด้วย ElastiCache Valkey แบบ Cache-Aside และ Content Caching (3) ตัวขับเคลื่อนกฎกติกาเกมแบบ Pure Function ที่รองรับคีย์เวิร์ด 14 ชนิดตามกฎหมายการ Comprehensive Rules v2.2.0 [1]

1.2 วัตถุประสงค์

1. แยกการบันทึกประวัติการแข่งขันและการคำนวณสถิติผู้เล่นออกจากกันด้วย SNS Fan-out และ SQS คู่ขนาน เพื่อให้ส่วนประกอบแต่ละส่วนทำงานและล้มเหลวได้อิสระ
2. เพิ่มความทนทานต่อข้อผิดพลาดที่ประมวลผลล้มเหลวด้วย Dead Letter Queue และ Redrive Policy
3. ลดเวลาตอบสนองของหน้าตารางผู้นำด้วย ElastiCache Valkey แบบ Cache-Aside
4. ลดขนาดการรับส่งไฟล์ข้อมูลการ์ดด้วย hash-based immutable asset และ gzip compression
5. สร้างตัวขับเคลื่อนกฎกติกาเกมที่ไม่มีผลข้างเคียง (Pure Function) รองรับคีย์เวิร์ด 14 ชนิดและระบบ Ability
6. ทดลองและรวบรวมหลักฐานการทำงานของระบบทุกส่วนบน AWS Academy Learner Lab

1.3 ขอบเขต

- ทดลองบน AWS Academy Learner Lab (us-east-1) ที่มีข้อจำกัด IAM Policy เฉพาะ (ไม่สามารถใช้ CloudFront, ElastiCache Serverless, Instance Refresh ได้)
- งบประมาณ credit รวม 50 ดอลลาร์สหรัฐ
- ข้อมูลการ์ดครอบคลุม Set 1 ถึง Set 13 จำนวน 3,242 ใบ
- ตัวขับเคลื่อนกฎกติกาฝั่ง client (browser) ไม่ได้ทำ authoritative validation บน server

บทที่ 2

ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

2.1 Amazon Simple Notification Service (SNS)

Amazon SNS เป็นบริการส่งข้อความแบบ Pub/Sub ที่ช่วยให้ผู้ส่ง (Publisher) ส่งข้อความไปยังหัวข้อ (Topic) เพียงครั้งเดียว แล้วผู้รับ (Subscriber) ทุกรายจะได้รับข้อความนั้นพร้อมกัน [2] ในโครงการนี้ใช้ SNS เพื่อกระจายผลการแข่งขันไปยังคิว SQS 2 คิวที่ทำหน้าที่ต่างกัน

2.2 Amazon Simple Queue Service (SQS) และ Dead Letter Queue

Amazon SQS เป็นบริการคิวข้อความที่รับประกันว่าข้อความจะถูกส่งอย่างน้อยหนึ่งครั้ง (at-least-once delivery) [3] เมื่อข้อความถูกประมวลผลไม่สำเร็จเกินจำนวนครั้งที่กำหนด (`maxReceiveCount`) Redrive Policy จะย้ายข้อความไปยัง Dead Letter Queue โดยอัตโนมัติ ป้องกันไม่ให้ข้อความเสียค่าระหว่างการประมวลผลข้อความอื่น

2.3 Amazon ElastiCache และรูปแบบ Cache-Aside

Amazon ElastiCache รองรับ Valkey (ซอฟต์แวร์ฐานข้อมูลในหน่วยความจำที่เข้ากันได้กับ Redis) [4] รูปแบบ Cache-Aside (Lazy Loading) ทำงานโดยแอปพลิเคชันตรวจสอบแคชก่อน หากไม่พบข้อมูล (Cache Miss) จึงดึงจากฐานข้อมูลหลักแล้วเขียนลงแคชพร้อมกำหนดอายุ (TTL) หากแคชหยุดทำงาน ระบบจะอ่านจากฐานข้อมูลโดยตรง (Graceful Degradation)

2.4 Content Caching ด้วย Hash-Based Immutable Assets

เมื่อ build tool (Vite) สร้างไฟล์ asset จะแทรก content hash ลงในชื่อไฟล์ (เช่น `lorcana_cards-DHNwgV-K.js`) ทำให้ทุกครั้งที่เนื้อหาเปลี่ยน ชื่อไฟล์จะเปลี่ยนตาม [5] เว็บเซิร์ฟเวอร์จึงตั้ง `Cache-Control: max-age=31536000, immutable` ให้เบราว์เซอร์เก็บแคชไว้ 1 ปีโดยไม่ต้องตรวจสอบซ้ำ ร่วมกับ gzip compression ที่ลดขนาดการรับส่ง

2.5 Node.js Cluster Module

Node.js รัน JavaScript บน event loop เดียว (single-threaded) [6] บน EC2 t3.micro ที่มี 2 vCPU หาก Node.js ทำงาน 1 process จะใช้ CPU ได้สูงสุดเพียง 50% ของเครื่อง ทำให้ Auto Scaling Policy ที่ตั้งเป้า 60% ไม่มีวันถูกกระตุ้น Cluster module แก้ปัญหานี้โดยสร้าง worker process 1 ตัวต่อ vCPU

2.6 Pure Function Game Engine

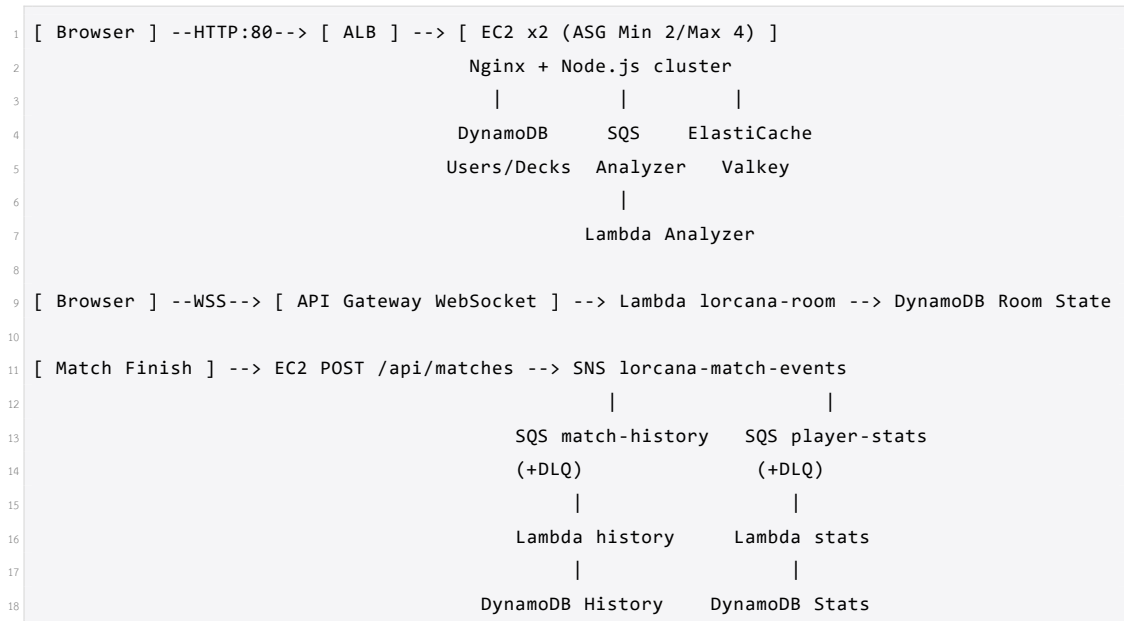
ตัวขับเคลื่อนกฎกติกาออกแบบเป็นฟังก์ชัน `applyAction(state, action)` ที่รับสถานะเกมปัจจุบันและการกระทำ แล้วคืนสถานะใหม่พร้อมรายการเหตุการณ์และ Prompt สำหรับเลือกเป้าหมาย โดยไม่เปลี่ยนแปลงสถานะเดิมและไม่เรียกใช้ React, Network

หรือ Browser API [7]

บทที่ 3

การออกแบบระบบ

3.1 สถาปัตยกรรมระบบรวม



Listing 3.1: แผนภาพสถาปัตยกรรมระบบ Disney Lorcana PlayLab Cloud (Stage 3)

3.2 เหตุผลในการออกแบบ (Design Rationale)

3.2.1 Multi-AZ โดยไม่ใช้ NAT Gateway

ระบบใช้ Public Subnet ใน 2 Availability Zones (**us-east-1a, us-east-1b**) โดยไม่ใช้ NAT Gateway เพื่อประหยัดค่าใช้จ่าย NAT Gateway มีค่าใช้จ่าย 0.045 ดอลลาร์ต่อชั่วโมง (ประมาณ 32.40 ดอลลาร์ต่อเดือน) ซึ่งเกินงบ credit 50 ดอลลาร์เพียงบริการเดียว

ข้อแลกเปลี่ยน: EC2 อยู่ใน Public Subnet จึงมี Public IPv4 (ค่าใช้จ่าย 0.005 ดอลลาร์ต่อชั่วโมงต่อ IP) ใช้ Chained Security Groups ควบคุมการเข้าถึงแทน โดย EC2 Security Group รับทราฟฟิกจาก ALB Security Group เท่านั้น ไม่เปิด inbound จาก **0.0.0.0/0** ค่า Public IPv4 จริงในเดือนกันยายน 2569 อยู่ที่ 1.96 ดอลลาร์ ซึ่งต่ำกว่า NAT Gateway ที่ประมาณ 32.40 ดอลลาร์ต่อเดือนอย่างมาก

3.2.2 Health Check ตรวจสอบเฉพาะ Node.js ไม่ตรวจสอบ DynamoDB

ALB Health Check (**GET /api/health**) ตรวจสอบเฉพาะว่า Nginx สามารถส่งต่อคำขอไปยัง Node.js ได้หรือไม่ โดยไม่ตรวจสอบการเชื่อมต่อ DynamoDB เหตุผลคือ DynamoDB เป็นบริการที่แชร์ร่วมกันระหว่างทุกเครื่อง หาก DynamoDB ชัดข้อง EC2 ทุกเครื่องจะถูกรับรู้เป็น unhealthy พร้อมกัน ทำให้ไม่มีเครื่องรับทราฟฟิกเลย ซึ่งเลวร้ายกว่าการให้เครื่องยังรับคำขอได้แต่คืน error เฉพาะคำขอที่ต้อง

ใช้ฐานข้อมูล

3.2.3 Node.js Cluster Mode สำหรับ t3.micro

t3.micro มี 2 vCPU แต่ Node.js single-thread ใช้ CPU ได้สูงสุด 50% ของเครื่อง Target Tracking Policy ที่ตั้งเป้า 60% จึงไม่มีวันถูกกระตุ้น การแก้ไขคือใช้ **cluster** module สร้าง 1 worker ต่อ vCPU (รวม 2 worker ต่อเครื่อง) ทำให้ CPU สามารถถูกใช้ได้เต็ม 100% และ Policy ทำงานตามที่ออกแบบ [6]

3.2.4 SNS Fan-out แทนการเขียน DynamoDB โดยตรง

เมื่อเกมจบ ระบบต้องบันทึกผลใน 2 ตาราง (ประวัติการแข่งขัน + สถิติผู้เล่น) หากเขียนโดยตรงใน endpoint เดียวกัน ความล้มเหลวของส่วนหนึ่งจะทำให้อีกส่วนไม่ถูกบันทึก การใช้ SNS กระจายข้อความไปยัง SQS 2 คิวคู่ขนาน ทำให้แต่ละส่วนล้มเหลวและ retry ได้อิสระ พร้อม DLQ สำรอง [2]

3.2.5 ElastiCache Node แทน Serverless

IAM Policy ของ AWS Academy Learner Lab มี explicit deny สำหรับ ElastiCache Serverless จึงต้องใช้ ElastiCache แบบ Node เดี่ยว (**cache.t3.micro**) ใน VPC Private Subnet ข้อจำกัดคือไม่มี automatic scaling และเป็น single point of failure แต่ระบบออกแบบ Graceful Degradation ให้อ่าน DynamoDB โดยตรงเมื่อแคชหยุดทำงาน

3.3 ตัวขับเคลื่อนกฎกติกาเกม

ตัวขับเคลื่อนแยกเป็น 5 module ใน **src/game/**:

ตารางที่ 3.1: โครงสร้าง Module ของตัวขับเคลื่อนกฎกติกาเกม

ไฟล์	ขนาด	หน้าที่
types.ts	3,262 B	นิยาม Interface: GameState, GameAction, ActionResult, LorcanCard
rules.ts	6,601 B	ฟังก์ชันตรวจสอบ: isCardLinkable, canPlayCard, canQuest, canChallenge
engine.ts	35,760 B	ฟังก์ชันหลัก applyAction รองรับ 16 action types
keywords.ts	4,102 B	parseCardKeywords ด้วย word-boundary regex 14 keywords
abilities.ts	5,936 B	parseAbility ตามจิ้งหะ 8 ประเภท, effect 10 ประเภท

บทที่ 4

ผลการทดลอง

การทดลองทั้งหมดดำเนินการวันที่ 27 และ 29 กันยายน 2569 บน AWS Academy Learner Lab (us-east-1) ไฟล์หลักฐานดิบอยู่ใน `docs/01_Reports/stage3_evidence/`

4.1 การทดลองที่ 1: การเล่นออนไลน์ข้ามเครื่อง

คำถาม: เมื่อมี EC2 2 เครื่อง ผู้เล่น 2 คนที่ ALB ส่งไปคนละเครื่องเล่นด้วยกันได้หรือไม่

วิธี: Playwright เปิด browser 2 context ผ่าน ALB ผู้เล่น 1 สร้างห้อง ผู้เล่น 2 เข้าด้วยรหัส 6 หลัก ตรวจสอบว่าผู้เล่น 1 เห็นชื่อผู้เล่น 2 ภายใน 10 วินาที

ผล: ผ่าน 5/5 รอบ ใช้เวลา 9.7–14.1 วินาทีต่อรอบ (รวม register, login, โหลดหน้า)

เหตุผลที่ต้องเปลี่ยน: เดิม WebSocket server บน EC2 เก็บ connection และ state ของห้องใน memory ของแต่ละเครื่อง ผู้เล่นที่อยู่คนละเครื่องจะไม่เห็นการเล่นของกัน ย้าย realtime ไป API Gateway WebSocket + Lambda ที่ใช้ DynamoDB เก็บ state แทน

4.2 การทดลองที่ 2: ความปลอดภัยบนระบบจริง

ตารางที่ 4.1: ผลทดสอบความปลอดภัย (OWASP A02, A05, A07)

Test Case	ผล
TC-AUTH-00a: backend รันโดยไม่มี secret	ก่อนแก้: รันต่อได้ (FAIL) / หลังแก้: exit 1 (PASS)
TC-AUTH-00c: รหัส admin ที่เขียนไว้ในโค้ด	ถูกปฏิเสธ (PASS)
TC-E2E-16: JWT ที่เซ็นด้วย secret default	ได้ 401 ทุกรอบ (10/10 รอบ)
TC-E2E-16: security headers	X-Frame-Options: DENY, X-Content-Type-Options: nosniff
Launch template	HttpTokens: required, ไม่มี secret ใน user data

Unit/integration test ทั้งหมด 128/128 ผ่าน (ณ วันที่ 30 กันยายน 2569)

4.3 การทดลองที่ 3: Failover เมื่อ Backend ล่ม

สคริปต์ `test_asg_recovery.ps1` สั่ง `systemctl stop lorcan-backend` บนเครื่องหนึ่งผ่าน SSM Run Command

ตารางที่ 4.2: ผลการทดลอง Failover (หลักฐาน: **failover_app_*.txt**)

เหตุการณ์	รอบ 1 (08:32:50)	รอบ 2 (11:22:57)
ASG สั่งเปลี่ยนเครื่อง	T+146 วินาที	T+90 วินาที
เครื่องใหม่ healthy	T+231 วินาที	T+166 วินาที
Alarm เป็น OK	T+428 วินาที	T+354 วินาที

ALB หยุดส่งทราฟฟิกไปเครื่องที่ล้มภายในราว 30 วินาที (health check 15 วินาที × 2 ครั้ง) ช่วง 30 วินาทีแรกนี้ request อาจได้ 502 เวลากู้คืน 2.8–3.9 นาที ส่วนใหญ่เป็นเวลาบูตเครื่องใหม่

4.4 การทดลองที่ 4: Auto Scaling

โหลดด้วย `POST /api/auth/login` (bcrypt cost 10 ใช้ CPU จริง) 40 worker ต่อเนื่อง

ตารางที่ 4.3: ผลการทดลอง Auto Scaling (หลักฐาน: **stress_scaling_*.txt**)

	รอบ 1 (process เดียว)	รอบ 2 (cluster 2 worker)
CPU เฉลี่ย	48.7–49.9% (ค้าง)	99.8–99.97%
Scale-out	ไม่เกิด	2 → 4 เครื่อง
Login สำเร็จ/ล้มเหลว	ไม่ได้บันทึก	9,653/0

4.5 การทดลองที่ 5: Infrastructure as Code

สคริปต์ `deploy_ec2_vpc_asg.ps1` สร้าง VPC, subnet 2 AZ, IGW, SG, ALB, TG, LT, ASG, scaling policy, alarm สำเร็จ
รัน `deploy` เข้าขณะ stack ยังอยู่หยุดด้วย `exit 1` `lab.ps1 destroy` ลบ resource ทั้งหมดเหลือแค่ default VPC

4.6 การทดลองที่ 6: Content Caching (T02)

ตารางที่ 4.4: ผล Content Caching (หลักฐาน: **caching_2026-09-29*.json**)

ตัวชี้วัด	ค่า
ขนาดเดิม (Uncompressed)	2,351,188 ไบต์ (2.35 MB)
ขนาดหลัง gzip	436,557 ไบต์ (426 KB) ลดลง 81.43%
เวลาโหลดครั้งแรก	602 มิลลิวินาที
เวลาโหลดครั้งที่สอง (Cache Hit)	86 มิลลิวินาที
ข้อมูลรับส่งครั้งที่สอง	0 ไบต์ (Memory Cache)

4.7 การทดลองที่ 7: Dead Letter Queue (T03)

ข้อความรูปแบบ JSON เสียหายถูกส่งเข้า SQS Lambda analyzer ส่งคืน **batchItemFailures** ข้อความผ่านการพยายามซ้ำครบ 3 ครั้ง (**maxReceiveCount: 3**) และเข้าสู่ DLQ สำเร็จที่ T+102 วินาที จำนวนข้อความค้างในคิวหลักหลังประมวลผล: 0 ข้อความ (หลักฐาน: **dlq_20260929_131824.log**)

4.8 การทดลองที่ 8: SNS Fan-out (T04)

เวลารวมของเทสต์ TC-E2E-23 (สมัคร, login, สร้างเด็ก, จบเกม แล้วรอจนผลปรากฏในประวัติและสถิติของผู้เล่นทั้งสองคน) คือ 4.9 วินาที จึงสรุปได้ว่า fan-out ใช้เวลาไม่เกิน 5 วินาทีหลังจบเกม (ยังไม่ได้แยกวัดเฉพาะช่วง SNS ถึง DynamoDB) (หลักฐาน: **decoupling_match_events_20260929.log**)

การป้องกันการเข้าถึง (OWASP A01): ส่งผลโดยไม่มี JWT ได้ 401 ส่งผลของแมตช์ที่ตนเองไม่ได้ร่วมเล่นได้ 403 การป้องกันซ้ำ (Idempotency): conditional write รูปแบบ **DEDUPE#<matchId>**

4.9 การทดลองที่ 9: ElastiCache Cache-Aside (T05)

วิธี: สคริปต์ **measure_leaderboard.mjs** เรียก **GET /api/leaderboard** ผ่าน ALB 8 รอบ แต่ละรอบรอให้ TTL 60 วินาทีหมดก่อน 1 request (ควรได้ MISS) แล้วยิงต่อเนื่อง 13 request (ควรได้ HIT) แยกกลุ่มตามค่า header **X-Cache** ที่ server ตอบจริง ค่า **durationMs** วัดภายใน backend ส่วนเวลาฝั่ง client รวมความหน่วงเครือข่ายจากเครื่องทดสอบไปยัง us-east-1

ตารางที่ 4.5: ผล ElastiCache Leaderboard เป็นค่ามัธยฐานและ p95 (หลักฐาน: **leaderboard_cache_2026-09-29T17-18-08-612Z.json**)

สถานการณ์	ตัวอย่าง	server (p95)	client (p95)
Cache Miss (อ่าน DynamoDB)	8	41 ms (112)	562.7 ms (814.2)
Cache Hit (ภายใน TTL 60 วินาที)	104	1 ms (2)	247.5 ms (255.9)

เมื่อแคชขัดข้อง (Connection refused) ระบบอ่าน DynamoDB โดยตรงและตอบ HTTP 200 (ทดสอบด้วย unit test TC-LEAD-04 และ TC-CACHE-004) เวลาประมวลผลฝั่ง server ลดจาก 41 เหลือ 1 มิลลิวินาที (ลดลง 97.56% เทียบค่ามัธยฐาน) เวลาที่ผู้ใช้เห็นลดจาก 562.7 เหลือ 247.5 มิลลิวินาที เพราะความหน่วงเครือข่ายราว 247 มิลลิวินาทีเป็นส่วนของแคชลดไม่ได้ กลุ่ม MISS มีเพียง 8 ตัวอย่างเพราะ TTL 60 วินาที ทำให้ต้องรอ 62 วินาทีต่อ 1 ตัวอย่าง (ครบกำหนด 100 ตัวอย่างต้องรอราว 100 นาที) และตารางสถิติมีข้อมูลจำนวนน้อย ค่า MISS บนตารางใหญ่กว่านี้จะสูงขึ้น

ปัญหาที่พบระหว่างทดลอง: รอบแรกได้ MISS ตลอดแม้เรียกซ้ำ สาเหตุคือ Redis client เชื่อมต่อตอนมี request แรก และตั้ง **enableOfflineQueue: false** ทำให้ request แรกของทุก process (2 เครื่อง x 2 worker) ล้มเจียบแล้วไปอ่าน DynamoDB แก่โดยเชื่อมต่อตอน server เริ่มทำงาน (**warmCache()**) และเพิ่ม unit test TC-CACHE-001 ถึง 004 หลังแก้ e2e TC-E2E-24 ผ่านได้ MISS แล้ว HIT ตามที่ออกแบบ

4.10 การทดลองที่ 10: Ability Coverage (T11)

สคริปต์ `measure_ability_coverage.ts` วิเคราะห์การ์ด 3,242 ใบ (Set 1–13) พบ ability 3,768 ข้อ (หลักฐาน: `ability_coverage.json`)

ตารางที่ 4.6: ระดับการรองรับ Ability ของตัวขับเคลื่อนกฎกติกา

ระดับ	จำนวน (%)	คำอธิบาย
Full Support	609 (16.16%)	ระบบคำนวณและเลือกเป้าหมายอัตโนมัติ
Partial Support	203 (5.39%)	ตรวจจับจังหวะ trigger ได้ ต้องระบุเพิ่มเติม
Manual Resolve	2,956 (78.45%)	แสดงป้ายให้ผู้เล่นจัดการเอง

บทที่ 5

ข้อจำกัดและค่าใช้จ่าย

5.1 ข้อจำกัดของระบบ

1. ไม่มี HTTPS: Learner Lab ไม่มีโดเมนจึงขอ certificate จาก ACM ให้ ALB ไม่ได้ ข้อมูล login จึงเป็น HTTP
2. WebSocket ไม่ตรวจ token: Lambda `lorcana-room` เชื้อชื่อผู้ใช้ที่ client ส่งมา
3. Rate limiter อยู่ใน memory ของแต่ละ process (OWASP A07): เมื่อมีหลาย worker และหลายเครื่อง ผู้โจมตีลองรหัสได้ $5 \text{ ครั้ง} \times \text{จำนวน worker} \times \text{จำนวนเครื่อง}$ แก้ไขได้ด้วย DynamoDB + TTL แต่เพิ่มค่าใช้จ่ายต่อ request
4. Secret ของ Lambda อยู่ใน environment variable แบบ plaintext (EC2 อ่านจาก SSM SecureString)
5. เวลาสุ่ม 2.8–3.9 นาที ขึ้นกับเวลาบูตเครื่องใหม่
6. Alarm มาช้ากว่าเหตุการณ์ 2–3 นาที เพราะ CloudWatch metric มาถึงช้า
7. ผลการแข่งขันรายงานจาก Client: ยังไม่ได้ประมวลผล state บน authoritative game server
8. CloudFront ใช้ไม่ได้: IAM Policy ปฏิเสธ `cloudfront:*`
9. ElastiCache Serverless ใช้ไม่ได้: IAM Policy มี explicit deny
10. Learner Lab session 4 ชั่วโมง: เครื่องถูก stop เมื่อหมด session
11. ค่าใช้จ่ายเมื่อเปิด 24 ชั่วโมงเกินงบ: ดูข้อ 5.2

5.2 ประมาณการค่าใช้จ่าย

ราคาต่อหน่วยเป็น on-demand list price ของ us-east-1 ตรวจสอบ ณ วันที่ 29 กันยายน 2569

ตารางที่ 5.1: ประมาณการค่าใช้จ่ายรายชั่วโมง

บริการ	ราคาต่อหน่วย	จำนวน	ต่อชม. (USD)
EC2 t3.micro	\$0.0104/ชม.	2	0.0208
EBS gp3 root 8 GiB	\$0.08/GB-เดือน	$2 \times 8 \text{ GiB}$	0.0018
Public IPv4	\$0.005/ชม.	4 (EC2 2 + ALB 2 AZ)	0.0200
ALB	\$0.0225/ชม. + LCU	1 (1 LCU)	0.0305
ElastiCache t3.micro	\$0.0170/ชม.	1 node	0.0170
Detailed monitoring	\$2.10/เครื่อง-เดือน	2	0.0058
CloudWatch alarm	\$0.10/alarm-เดือน	3	0.0004
SNS, SQS, Lambda, DynamoDB	ระดับทดสอบ	–	≈ 0.0000
รวม			≈ 0.096

ตารางที่ 5.2: ค่าใช้จ่ายจริงจาก AWS Billing (1-27 กันยายน 2569)

บริการ	ก.ย. 2569 (USD)	สัดส่วน
Elastic Load Balancing	7.13	69%
Amazon VPC (public IPv4)	1.96	19%
CloudWatch	0.45	4%
EC2 Compute	0.44	4%
EC2 Other (EBS)	0.31	3%
รวม month-to-date	10.29	

ALB เป็นค่าใช้จ่ายหลักเพราะคิดรายชั่วโมงตลอดเวลาที่ยังไม่ลบ แม้ EC2 จะถูก scale เป็น 0 แล้ว หากหยุดทดสอบเกิน 1 วัน ควรสั่ง `lab.ps1 destroy` เพื่อป้องกัน credit หมด

บทที่ 6

สรุปและข้อเสนอแนะ

6.1 สรุปผลการดำเนินงาน

ระบบ Disney Lorcana PlayLab Cloud ในระยะที่ 3 ขยายจากสถาปัตยกรรมพื้นฐานของระยะที่ 2 ด้วยส่วนประกอบ 5 ส่วน:

1. Content Caching ลดขนาดการรับส่งไฟล์ข้อมูลการ์ด 81.43% และเวลาโหลดซ้ำจาก 602ms เหลือ 86ms
2. Dead Letter Queue จัดการข้อความเสียหายได้ภายใน 102 วินาทีโดยไม่ส่งผลกระทบต่อคิวหลัก
3. SNS Fan-out กระจายผลการแข่งขันไป 2 คิวคู่ขนานภายใน 5 วินาที พร้อมป้องกันการนับซ้ำ
4. ElastiCache Cache-Aside ลดเวลาประมวลผลฝั่ง server ของ Leaderboard จาก 41 ms เหลือ 1 ms (ค่ามัธยฐาน ลดลง 97.56%) พร้อม Graceful Degradation เมื่อแคชหยุดทำงาน
5. ตัวขับเคลื่อนกฎกติกา Pure Function รองรับ 16 action types, 14 keywords, ระบบ Ability 10 ประเภท, Shift, Location, Song และ Item ผ่านชุดทดสอบอัตโนมัติ 128 รายการ

การทดลอง 10 หัวข้อยืนยันว่าระบบทำงานตามที่ออกแบบภายใต้ข้อจำกัดของ AWS Academy Learner Lab ค่าใช้จ่ายจริง 10.29 ดอลลาร์ (27 วัน) อยู่ภายในงบ 50 ดอลลาร์

6.2 ข้อเสนอแนะ

1. ย้าย rate limiter จาก in-memory ไปใช้ DynamoDB + TTL เพื่อแก้ปัญหา OWASP A07
2. เพิ่ม authoritative game state validation บน server เพื่อป้องกันผลการแข่งขันที่ถูกแก้ไขจาก client
3. หากมีโดเมนจริง ควรเพิ่ม HTTPS ผ่าน ACM + ALB listener 443
4. ขยาย Full Support ของ ability จาก 16.16% ให้ครอบคลุมกลุ่ม timing ที่พบบ่อย (activated 412 ข้อ, on_play 596 ข้อ)
5. พิจารณาใช้ CloudFront เป็น CDN หาก deploy นอก Learner Lab

บรรณานุกรม (References)

- [1] Ravensburger & Disney. (n.d.). *Disney Lorcana Comprehensive Rules v2.2.0*. [Online]. Available: <https://www.disneylorcana.com/resources> (ไฟล์ที่ใช้อยู่ที่ docs/Rule/Comprehensive-Rules_2.2.0-EN.pdf)
- [2] Amazon Web Services. (2026). *Amazon SNS and SQS Fanout Pattern*. AWS Architecture Center. [Online]. Available: <https://docs.aws.amazon.com/sns/latest/dg/sns-sqs-as-subscriber.html>
- [3] Amazon Web Services. (2026). *Amazon SQS Dead-Letter Queues*. Amazon SQS Developer Guide. [Online]. Available: <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html>
- [4] Amazon Web Services. (2026). *Amazon ElastiCache for Valkey*. [Online]. Available: <https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/WhatIs.html>
- [5] Vite. (2026). *Building for Production: Asset Hashing*. [Online]. Available: <https://vite.dev/guide/build>
- [6] OpenJS Foundation. (2026). *Cluster Module*. Node.js Documentation. [Online]. Available: <https://nodejs.org/api/cluster.html>
- [7] Amazon Web Services. (2026). *Target Tracking Scaling Policies for Amazon EC2 Auto Scaling*. [Online]. Available: <https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-target-tracking.html>
- [8] Amazon Web Services. (2026). *Health Checks for Application Load Balancer Target Groups*. Elastic Load Balancing User Guide. [Online]. Available: <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-health-checks.html>
- [9] Amazon Web Services. (2026). *AWS Systems Manager Parameter Store*. [Online]. Available: <https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html>
- [10] OWASP Foundation. (2021). *OWASP Top 10: 2021*. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [11] Amazon Web Services. (2026). *Amazon VPC Pricing (Public IPv4 Charges)*. [Online]. Available: <https://aws.amazon.com/vpc/pricing/>